From Tool Invocation to Source-Mechanism Exploration: Protected White-Box DSE for Open-Source EDA

Zhiyu Zheng
Fudan University
Shanghai, China
zyzheng24@m.fudan.edu.cn

Yiming Du
Fudan University
Shanghai, China
iiamadoga@163.com

Ziyi Wang
The Chinese University of Hong Kong
Hong Kong, China
zeayw1@gmail.com

Zhiang Wang
Fudan University
Shanghai, China
zhiangwang@fudan.edu.cn

Abstract

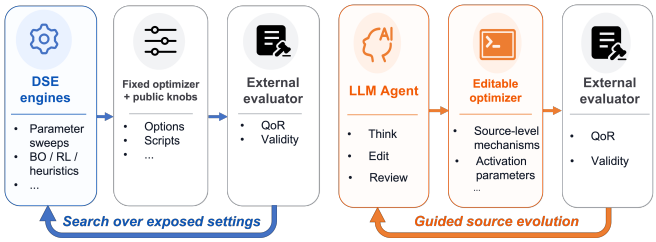
Open-source EDA tools allow design-space exploration (DSE) to move beyond public knobs and into bounded source-level mechanisms inside staged optimizers. We present ReviewDSE, a protected white-box DSE framework that explores such mechanisms for a target design. ReviewDSE evaluates complete source candidates under a protected evaluator and records reusable search knowledge as reviewed mechanism-level evidence. It first constructs method evidence and source-start branches from calibration designs, then uses these fixed warm-start products to initialize target-case exploration under Teacher review and full-flow validation. We instantiate ReviewDSE on OpenROAD detailed placement as a representative staged open-source EDA optimizer. Across nine target tasks, ReviewDSE reduces final post-DPL half-perimeter wirelength (HPWL) by 1.78% on average under a 2× runtime gate, compared with 0.38% for public-knob black-box DSE. A runtime-aware ReviewDSE selection retains a 1.68% reduction at 1.11× runtime, and full-flow review exposes stage-composability failures while source-mechanism exploration repairs hard cut-row legality failures.

1 Introduction

Design-space exploration (DSE) is a standard methodology for improving electronic-design-automation (EDA) flows [1]. Conventional DSE treats each tool as a fixed executable and searches its public control surface, such as command options, flow scripts, or other exposed knobs, to find configurations with better quality of results [2]. This black-box formulation is practical and broadly applicable, especially for closed-source tools, but it leaves quality-critical implementation choices inside large EDA codebases outside the search space.

Open-source EDA changes where the DSE boundary can be drawn. In an open codebase such as OpenROAD [3], an optimizer is not only invocable but also inspectable and modifiable. This enables white-box DSE for open-source EDA: instead of searching only over public knobs of a fixed executable, the search can enter selected parts of the optimizer implementation and explore bounded source-level mechanisms. In EDA flows, this mechanism space is naturally organized by stages and stage interfaces. A placement flow, for example, may include legalization policies, local improvement kernels, objective scoring, displacement control, rollback logic, and state passed from legalization to detailed-placement optimization. White-box DSE therefore asks which internal mechanisms should be changed, where they should be inserted, and how they should be configured for a target design.

The expanded mechanism space is difficult to explore manually. Source-level choices are numerous, implementation-dependent, and strongly coupled across stages. A local improvement in one stage may not survive the downstream flow; for example, a legalizer that reduces post-legalization HPWL can still produce a placement that is harder for detailed-placement optimization (DPO) to improve. Recent coding-evolution systems suggest a possible path forward: language-model agents can modify programs using evaluator feedback [4–6], and related ideas have begun to appear in EDA source-level evolution [7–10]. These systems show that agents can generate useful code changes for complex tools.



(a) Conventional Black-box DSE (b) White-box DSE

Figure 1: Search boundary for one target: black-box DSE tunes public knobs; white-box DSE explores bounded internal mechanisms under the protected evaluator.

Source modification alone does not make a reliable DSE procedure. Existing loops often treat each generated diff as a complete variant to be selected or rejected as a whole. This whole-variant view gives limited reusable evidence about which mechanism caused a gain, which stage activated it, whether it was live, whether it was runtime-expensive, or why it failed to compose with downstream optimization. Unconstrained source editing can also corrupt the evaluator path, change scripts or parsers, bypass legality checks, or produce stale source/binary/metric mismatches. Reliable white-box DSE therefore needs both a protected evaluator and a review unit finer than the complete diff.

We present ReviewDSE, an agentic white-box DSE framework for open-source EDA. ReviewDSE evaluates complete source candidates under the protected evaluator, while recording reusable search knowledge as reviewed stage/interface mechanism evidence. This evidence captures source references, liveness signals, compatibility observations, and promotion or rejection decisions. ReviewDSE uses a Teacher–Student agent structure: the Teacher reviews full-flow behavior and mechanism evidence, while Student agents adapt source starts, borrow peer mechanisms, or repair failed edits.

ReviewDSE follows a two-level workflow. Level 1 builds method evidence and source-start branches from calibration designs. Level 2 conducts target-design exploration through initialization based on the frozen evidence from Level 1, followed by iterative search driven by target-specific metrics, logs, diffs, peer candidates, and Teacher review. We instantiate ReviewDSE on OpenROAD detailed placement,

where legalization, DPO, and their handoff expose strict legality and stage-composability trade-offs.

This paper makes the following major contributions:

- We formulate protected white-box DSE for open-source EDA, where the search space extends from public tool controls to bounded source-level mechanisms inside staged optimizers.
- We introduce ReviewDSE, a two-level source-mechanism exploration framework that builds calibration-time method evidence and source-start branches, then uses them to initialize target-specific exploration with Teacher review, liveness checks, full-flow validation, and target-local evidence.
- We instantiate ReviewDSE on OpenROAD detailed placement and show that reviewed source-mechanism exploration improves the post-DPL HPWL/runtime frontier beyond public-knob BO-DSE, exposes why stage-local winners require full-flow review, and recovers legality on hard cut-row stress tests.

The rest of the paper is organized as follows. Section 2 reviews related work. Section 3 presents ReviewDSE. Section 4 reports the results. Sections 5 and 6 discuss limitations and conclude the paper.

2 Background and Related Work

2.1 DSE in EDA

Design space exploration (DSE) is widely used in EDA because QoR is highly sensitive to algorithm choices, tool parameters, and flow-stage interactions [1]. Prior work has explored DSE at multiple abstraction levels. In high-level synthesis, AutoDSE [11] searches HLS pragmas and transformations through bottleneck-guided exploration. In logic synthesis, BOiLS [12] formulates synthesis-flow selection as a sequential optimization problem and searches logic-optimization recipes with Bayesian optimization. In physical design, placement-tuning methods optimize placement parameters and objectives using learning-based or multi-objective search [13, 14]. These works show that automated search can improve QoR and reduce manual effort across EDA flows.

However, most existing EDA DSE methods remain black-box with respect to the optimizer implementation. Their search spaces are defined by exposed pragmas, parameters, command sequences, scripts, seeds, or flow-level configurations, while the tool source remains fixed. Such methods can improve how tools are invoked, but cannot directly explore internal mechanisms such as hidden cost models, repair policies, rollback decisions, or stage interfaces. Open-source infrastructures such as OpenROAD [3] make these implementation-level decisions searchable under the same external flow metrics, motivating white-box DSE where bounded source-level mechanisms become design choices.

2.2 LLM Agents for EDA

Large language models have recently been explored in EDA as natural-language interfaces, code generators, and tool-assistance agents. One major line of work studies RTL or Verilog generation from natural-language specifications, with benchmarks such as RTLLM [15] and VerilogEval [16] evaluating syntax, functional correctness, and design quality, and domain-specific models such as VeriGen [17] and RTLCoder [18] improving HDL generation through EDA-specific training. Beyond direct code generation, feedback-driven frameworks use compiler or simulation outputs to repair generated HDL, as in AutoChip [19]. LLMs have also been used for hardware verification and testbench generation [20, 21]. Another line of work uses LLMs as EDA tool assistants: ChatEDA [22] supports task decomposition, script generation, and tool execution, ChipNeMo [23] studies domain-adapted assistants for chip-design tasks, and documentation-grounded systems answer questions over EDA manuals and OpenROAD documentation [24, 25]. These works improve how designers generate artifacts, query documentation, and invoke EDA tools, but they mostly operate around a fixed executable.

Table 1: Positioning of ReviewDSE.

Category	Search point	Review signal	Search memory
EDA parameter DSE	knobs/scripts	QoR logs	best settings
LLM tool use	artifacts/scripts	tool logs	revised artifact
Source evolution	source patch	build/QoR logs	winning patch
ReviewDSE	source mechanism edit	protected full-flow evidence	reusable mechanism evidence

Recent coding-evolution systems show that LLM agents can modify programs using evaluator feedback [4–6]. Similar ideas have begun to appear in EDA source-level evolution [7–10]. In this setting, the LLM action moves from scripts or artifacts around the tool to the implementation of the tool itself. Self-Evolved ABC [9] evolves a logic-synthesis codebase under correctness and synthesis-QoR feedback. AuDoPEDA [10] produces repository-grounded OpenROAD diffs using code-graph documentation and QoR-driven execution. GR-Evolve [8] specializes global-router source code to individual designs, and EvoPlace [7] evolves global-placement optimization components.

These works establish that LLM agents can improve EDA tools through source modification. ReviewDSE addresses a complementary question: how to make source modification a reliable white-box DSE procedure. Rather than treating each generated diff only as a whole-program winner or loser, ReviewDSE evaluates complete candidates under a protected flow while recording reusable positive and negative evidence at the mechanism level. This allows useful calibration evidence, stage donors, and rejection decisions to guide later search rounds. Table 1 summarizes the position of ReviewDSE relative to prior directions. ReviewDSE still evaluates complete source candidates, but its reusable memory is mechanism-level evidence that later agents can refine or combine during white-box optimization.

3 Methodology

3.1 Formulation and Overview

Conventional black-box DSE treats the EDA optimizer as a fixed executable evaluated by an external flow-level evaluator. Let O denote the implementation of a staged optimizer for a target design d, let E denote the protected evaluator used to run the flow and collect canonical metrics, and let $x \in \mathcal{X}$ denote the public configuration vector exposed through command options, flow scripts, pass sequences, or other tool parameters. Black-box DSE searches over $x \in \mathcal{X}$. Each search point is evaluated as $E(O, x, d)$, while the optimizer implementation O and the measurement path E remain fixed. This formulation is practical, but it limits the search to behaviors already exposed by the tool developer.

ReviewDSE keeps the same evaluator E but expands the search variable from a public configuration to a bounded source edit. Let $\mathcal{S}$ denote the editable optimizer source surface and let $\mathcal{D}(\mathcal{S})$ denote the

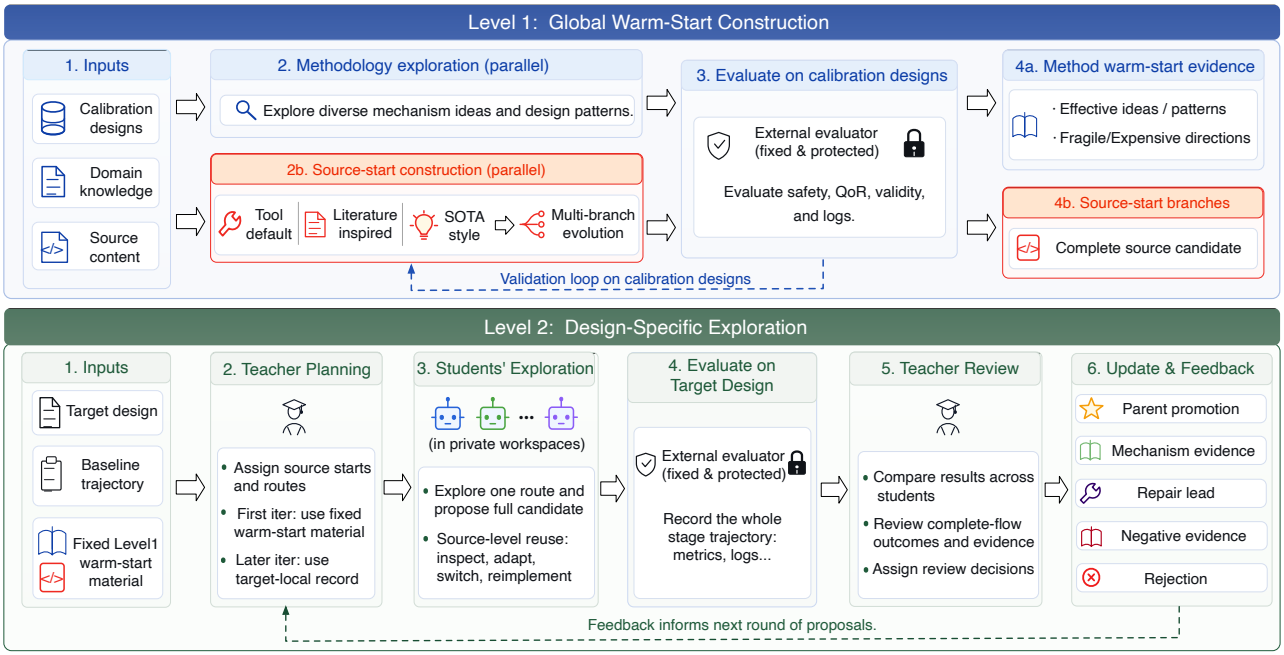


Figure 2: ReviewDSE workflow. Level 1 builds fixed warm-start evidence and source-start branches. Level 2 initializes target search from them, then explores candidates under the protected evaluator.

admissible edits on that surface. ReviewDSE searches over $\Delta \in \mathcal{D}(\mathcal{S})$. Each search point produces a complete source candidate $O+\Delta$, which is built and evaluated on d as $E(O+\Delta, d)$.

EDA optimizers are naturally organized as staged flows, with intermediate states passed across neighboring stages. ReviewDSE uses this native structure to define review scopes for source edits. Let $\mathcal{C}$ denote the set of stage and interface scopes, where each $c \in \mathcal{C}$ corresponds to either an existing optimization stage or a handoff between two neighboring stages. Each scope has a pool of candidate mechanisms,

$$\mathcal{M}_c = \{p_{c,1}, \ldots, p_{c,n_c}\}. \tag{1}$$

A mechanism is not the stage itself. It is a source-level logic chain that changes how the stage initializes state, generates candidates, scores moves, accepts or rolls back transactions, exports handoff state, or performs local repair. A mechanism may also be controlled by activation parameters $\theta_{c,j}$, such as weights, thresholds, budgets, seeds, fallback rules, or early-stop conditions.

A candidate edit Δ may touch one or more native scopes. For each scope c, let $\mathcal{M}_c(\Delta) \subseteq \mathcal{M}_c$ be the mechanisms implemented by the source candidate produced by Δ. The mechanism content of the candidate can be written as

$$\mathrm{mech}(\Delta) = \bigcup_{c \in \mathcal{C}} \{(c, p_{c,j}, \theta_{c,j}) \mid p_{c,j} \in \mathcal{M}_c(\Delta)\}. \tag{2}$$

This notation describes the mechanisms realized by an implemented source candidate. It does not imply that ReviewDSE materializes standalone mechanism units and assembles them into a tool variant. The actual search point is a source workspace built and evaluated as $O + \Delta$. After evaluation, ReviewDSE retains mechanism-level evidence rather than reusable code objects. As summarized in Table 2, this evidence records source anchors, liveness signals, full-flow behavior, and review outcomes, allowing later search to inspect, adapt, or reimplement the reviewed mechanism logic in another candidate. A record may indicate that the mechanism improves the target, is active but harmful, never executes, depends on a compatible handoff, or should be avoided.

Table 2: Reviewed mechanism evidence record.

Field	Recorded information
Scope	Target stage or stage interface, used to relate evidence to stage-wise behavior.
Source reference	Branch or patch reference, modified files, code anchors, and mechanism intent.
Controls/liveness	Activation parameters, counters, traces, and artifacts showing execution.
Review outcome	Full-flow metrics, compatibility observations, and Teacher decisions such as parent promotion, mechanism evidence, repair lead, negative evidence, or rejection.

3.2 Protected Evaluation Contract

The protected evaluator is the trust boundary for source-level DSE. It fixes the benchmark inputs, incoming design state, top-level command sequence, baseline harness, legality or validity checker, metric parser, reference comparison, runtime gate, evaluator scripts, and unrelated tool components. Evaluator-produced objective and stage metrics remain outside the editable source surface. A candidate is rejected if it changes or bypasses this path, fails to build, violates validity checks, misses canonical metrics, exceeds the runtime gate, or lacks liveness evidence for its claimed mechanism.

Figure 2 summarizes how ReviewDSE works. Level 1 constructs frozen warm-start evidence before target optimization: method evidence records calibration-observed mechanism routes, and source-start construction produces complete source basins. Level 2 uses this evidence as prior guidance, but every promoted candidate is rebuilt, executed, and reviewed on the current target case. Level 2

Table 3: Teacher review decisions used in exploration.

Decision	Condition	Use
Parent promotion	Valid, complete metrics, live, within gate, improves target score	Next parent
Mechanism evidence	Active mechanism helps a stage or interface but loses final score	Evidence record
Repair lead	Explains validity/runtime failure or fragile handoff	Targeted route
Negative evidence	Inactive, too expensive, or non-composable	Search constraint
Candidate rejection	Build, validity, metric, stale-binary, or evaluator-path failure	Discard

may accumulate target-local evidence, but it does not update the global warm-start evidence.

3.3 Calibration Warm-Start Construction

Level 1 prepares the search by building two fixed warm-start products before any target case is optimized: method warm-start and source-start branches. It does so by using calibration designs as diagnostic probes. These designs are not target designs. Instead, they are constructed to expose different optimizer behaviors, including stage-local damage, downstream recoverability, constraint pressure, runtime pressure, and stage-interface incompatibility. Level 1 converts these observations into mechanism hypotheses and executable starting points, so that Level 2 does not begin from a blank prompt.

Method warm-start construction focuses on the hypothesis side. Agents inspect papers, source code, and domain knowledge, then implement candidate mechanisms on existing branches and evaluate them with the protected flow. Level 1 review summarizes the calibration behavior of each idea, including whether it is promising, fragile, too expensive, inactive, or non-composable. The resulting records provide route and diagnosis evidence for the Level 2 Teacher when a target case shows similar stage movement or design characteristics.

Source-start construction focuses on the implementation side. Some literature-inspired or SOTA-style ideas are too large to recreate in each target iteration, so Level 1 materializes them as complete source branches during calibration. In Level 2, these branches serve as editable starts rather than certified target solutions. Students may extend one, switch to another, or reimplement a useful logic chain after inspecting peer diffs. This gives the target search concrete code for implementing or repairing a route suggested by the method evidence.

3.4 Target-Case Source Exploration Loop

Level 2 runs an independent white-box optimization loop for each target design. For a new target, ReviewDSE first measures the target baseline trajectory and creates private Student workspaces. The loop maintains a local record that includes baseline metrics, evaluated candidates, stable source references, implementation diffs, Student knowledge cards, Teacher reviews, peer-learning packets, and any clean promoted parent. The Level 1 warm-start material is only used as prior knowledge, and subsequent decisions are driven by the local record accumulated on the current design.

In the first iteration, the Teacher compares the target baseline trajectory and design characteristics with the fixed Level 1 method evidence and available source branches. Based on this comparison, the Teacher assigns each Student an initial source start and mechanism route, with generated packets and route tables as supporting evidence. This assignment only defines the starting direction for

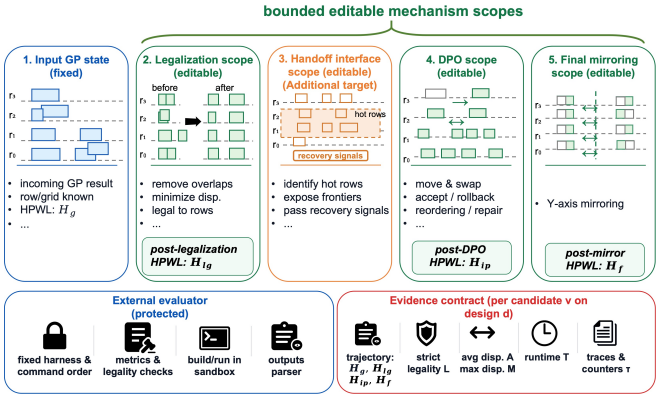


Figure 3: OpenROAD DPL instantiation. ReviewDSE edits bounded legalization, DPO, and handoff mechanisms while keeping the evaluation path protected.

the target search. The selected branch is not fixed across iterations. Later rounds may extend it if it remains effective, or switch to repairs, alternative branches, or new mechanisms when target-local evidence suggests a better route.

In later iterations, the Teacher first reviews the previous round and then decides how to balance continuation and exploration. A clean candidate with the best final objective inside the runtime gate may seed an elite continuation branch, typically for one Student. The other Students can explore diversity routes or reuse evidence from peer candidates. This reuse remains source-level: a Student may inspect another candidate's diff, summarize its logic chain, adapt a patch, switch to a branch, or reimplement the idea in its own workspace. The reviewed mechanism guides the edit, but it is not copied as an independent plug-in.

Each Student edits a private optimizer source workspace and submits a complete $O + \Delta$ candidate. The Student must build or relink a private binary, run the fixed evaluator with that binary, and provide complete artifacts. Because earlier-stage choices affect later stages, the Student is encouraged to reason about the whole stage trajectory rather than optimizing a stage metric in isolation.

After evaluation, the Teacher reviews the complete-flow outcome together with the mechanism evidence behind it, and assigns a review decision following Table 3. A valid improved candidate may become the promoted parent for later target search. Other evaluated candidates may still be retained as target-local evidence when their source diffs and logs explain an active mechanism, even if they are not promoted. Invalid candidates are recorded but cannot be promoted. These review outcomes update only the target-local record, while the Level 1 warm-start evidence remains unchanged.

3.5 Instantiation on OpenROAD Detailed Placement

We instantiate ReviewDSE on OpenROAD detailed placement.[1] We refer to the source-specialized commands as DPL-evolve, DPO-evolve, and mirror-evolve. Figure 3 summarizes the editable and protected parts of this instantiation. The evaluated command sequence is fixed and keeps the canonical three stages: legalization, DPO, and mirroring. ReviewDSE edits only the bounded source implementations

[1]We use constructed Nangate45 detailed-placement calibration cases for Level 1: JPEG at UTIL=90, AES at UTIL=70, and a SWERV wrapper at UTIL=60.

invoked by these stages in private workspaces. The strict comparison anchor is the fixed OpenROAD detailed-placement flow.

Following the formulation above, we use legalization and DPO as the two main stage scopes, and we use the legalization-DPO handoff as the main interface scope. In legalization, source-start branches include Diamond-style, Negotiation-based, and LEGALM-inspired methods. Within this scope, ReviewDSE may explore row-assignment policies, row-segment reservation, overflow-aware relocation, connectivity-aware escape, bounded compaction, and post-legality repair.

DPO exploration starts from the same native improve-placement source inside the selected legalizer branch. The Teacher assigns the mechanism implementation path, and the Student edits the relevant DPO source path. The native improve-placement behavior contains local kernels such as independent-set matching, global swaps, vertical swaps, row-local reordering, randomized local improvement, and legality or orientation checks. ReviewDSE may revise or extend this source with local-move generation, partner selection, region kicks, dirty-window dynamic programming, exact affected-net HPWL scoring, transaction accept/rollback, fallback descent, per-net extrema caches, and thread-local scratch storage. Handoff mechanisms include dirty rows, pressure regions, touched-net frontiers, boundary-active cells, local repair windows, and source-produced state that downstream DPO can consume.

For an evaluated candidate $O+\Delta$ on design d, the evaluator records the full stage trajectory

$$E(O + \Delta, d) = \{H_g, H_{lg}, H_{ip}, H_f, L, A, M, T, \tau\}, \tag{3}$$

where H_g, H_{lg}, H_{ip}, and H_f are half-perimeter wirelength (HPWL) after incoming global placement, legalization, DPO, and final post-DPL evaluation. L denotes strict legality, A and M are average and maximum displacement, T is runtime, and τ contains traces, liveness counters, and diagnostics. Full-trajectory review is necessary because a stage-local improvement is not always a valid target improvement. A lower legalization HPWL H_{lg} may still worsen the final post-DPL HPWL H_f after DPO and mirroring. Conversely, a non-winning candidate can also provide active mechanism evidence under specific target patterns. This is why ReviewDSE retains mechanism evidence for later target search.

4 Experiments

Our experiments compare white-box source optimization with black-box DSE, test runtime-aware selection, examine full-flow composability, and evaluate whether source-mechanism exploration can recover legality on hard cut-row layouts.

4.1 Experiment Setup and Metrics

We use a fixed OpenROAD/OpenROAD-flow-scripts checkout in the protected evaluator and run all methods through the same measurement path. The exact checkout, evaluator scripts, and selected-candidate artifacts are recorded with the experiment data. The main target search uses one Teacher agent with the GPT-5.5 xhigh profile and four parallel Student agents with the GPT-5.4 xhigh profile for 10 review iterations. We report only candidates that compile, pass strict legality, produce complete protected metrics, satisfy source/binary/metric consistency, show liveness, and stay within a 2× runtime gate.

Table 4: Nine-case DSE results.

Design	Default	BO	ReviewDSE-HPWL	ReviewDSE-G_{HR}	Tokens
AES ASAP7	42.2k/5.3s	-0.98%/1.47×	-1.91%/1.91× i10	-1.40%/1.38× i10	1.62/0.08
AES N45	176.8k/6.0s	-1.20%/1.38×	-3.67%/1.86× i10	-3.67%/1.86× i10	1.81/0.09
Ariane133 N45	5.78M/151s	-0.06%/0.92×	-5.50%/0.56× i10	-5.50%/0.56× i10	3.66/0.11
Ibex ASAP7	63.3k/6.9s	-0.10%/1.10×	-0.60%/1.17× i10	-0.60%/1.17× i10	1.68/0.10
Ibex N45	200.1k/5.1s	-0.09%/1.00×	-0.99%/2.00× i10	-0.56%/1.08× i6	1.52/0.12
JPEG ASAP7	134.6k/23s	-0.63%/1.39×	-0.94%/1.00× i5	-0.94%/1.02× i3	1.87/0.10
JPEG N45	467.2k/33s	-0.23%/1.13×	-1.63%/0.56× i9	-1.63%/0.77× i10	1.93/0.09
SWERV ASAP7	913.9k/57s	-0.04%/1.08×	-0.33%/1.90× i8	-0.30%/1.09× i4	3.06/0.09
SWERV N45	3.27M/34s	-0.09%/1.08×	-0.49%/1.07× i9	-0.49%/1.07× i9	2.14/0.14
Mean	–	-0.38%/1.17×	-1.78%/1.34×	-1.68%/1.11×	2.15/0.10

Entries are post-DPL HPWL delta/runtime vs. default; *i#* is the selected iteration. Tokens are logged/active budgets in billions.

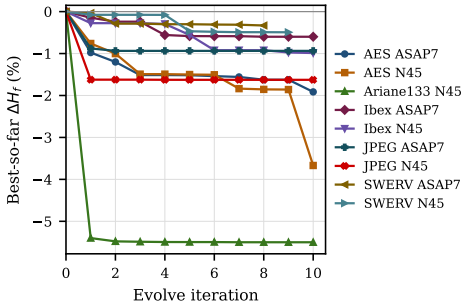


Figure 4: 10-iteration best-so-far ΔH_f.

We compare against BO-DSE, a representative black-box DSE baseline that keeps the OpenROAD source fixed. BO-DSE uses Bayesian optimization (BO) with Optuna's Tree-structured Parzen Estimator (TPE) sampler. It searches exposed legalization, DPO, extra-invocation, and global-swap controls for 400 trials per case with four-way parallelism.

Our experimental dataset comprises nine target tasks derived from the AES, Ariane133, Ibex, JPEG, and SWERV designs, implemented in ASAP7 and Nangate45 technology nodes. The calibration and target sets are disjoint evaluation instances. They may share benchmark families, but no target placement, target metric, target route, or target-specific candidate is used in Level 1. All target designs are evaluated independently by the protected flow.

To evaluate quality of results (QoR), we use the final post-DPL HPWL, denoted as H_f, as the primary metric. We report the relative HPWL change ΔH_f and runtime compared with the default Diamond detailed-placement flow. For ReviewDSE, each test case reports both the best-H_f candidate that satisfies the 2× runtime gate and a lower-overhead candidate selected by a runtime-aware score G_{HR}[2].

4.2 White-Box Source Optimization

Table 4 compares the OpenROAD default, public-knob BO-DSE, and ReviewDSE on the nine target tasks. After 400 trials per case, BO-DSE improves mean post-DPL HPWL by 0.38%, while ReviewDSE improves all nine targets by 1.78% on average within the same 2×

[2]The higher-is-better selection score is defined as $G_{HR} = 100 \cdot (H_f^{Default} - H_f)/H_f^{Default} - P(T/T^{Default})$, where T is the runtime, $P(r) = 0$ for $r \le 1.10$, and $P(r) = (\sqrt{r} - \sqrt{1.10})/(\sqrt{2} - \sqrt{1.10})$ for $r > 1.10$. A 2× runtime penalty requires a one percentage point reduction in post-DPL HPWL to break even; small runtime noise is ignored.

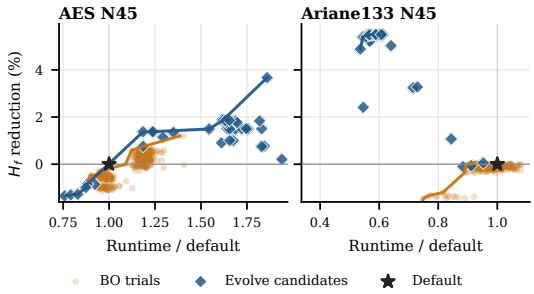


Figure 5: Runtime–quality frontier on two Nangate45 cases; y-axis is post-DPL HPWL reduction vs. default.

runtime gate. Runtime-aware ReviewDSE-G_{HR} preserves a 1.68% mean HPWL reduction while reducing average runtime from 1.34× to 1.11×. This comparison uses the same protected evaluator and target tasks, but the two methods search different spaces: BO-DSE searches exposed controls of a fixed source tree, while ReviewDSE explores reviewed source mechanisms. Calibration evidence only initializes promising source directions. Every reported candidate is rebuilt, evaluated, and re-validated on its own target by the protected evaluator. The token column reports mean per-target search cost.

Figure 4 shows best-so-far ΔH_f over 10 target iterations. Early drops indicate that Level 1 evidence can choose useful initial source directions.[3] Later drops, such as AES N45, reflect target-specific source exploration after Teacher review, runtime feedback, and peer-candidate evidence. Flat segments correspond to iterations without a promoted parent, often because review rejects non-composable or runtime-expensive candidates.

Figure 5 plots HPWL reduction against normalized runtime for two representative Nangate45 cases. On AES N45, ReviewDSE reaches a 3.67% HPWL reduction and outperforms the best BO-DSE point at comparable runtime. On Ariane133 N45, ReviewDSE reaches a 5.50% HPWL reduction while reducing runtime relative to the default. These tradeoffs were not observed in the 400-trial BO-DSE search over the specified public-knob space.

4.3 Full-Flow Review and Downstream Composability

We next examine why ReviewDSE reviews complete trajectories instead of accepting stage-local winners. The test applies a deliberately stage-local selection rule: it selects candidate legalizers by post-legalization HPWL H_{lg}, then passes the selected placements through the same downstream DPO and final mirroring flow. The dense N45 cases are constructed validation variants with increased legalization pressure and are not included in the nine-case QoR average. Table 5 reports cases where the selected legalizer improves H_{lg} but degrades the final post-DPL HPWL H_f.

These counterexamples show that a lower-HPWL legalized placement can be less compatible with downstream DPO. JPEG dense N45 is the clearest case: the selected legalizer reduces H_{lg} by 14.96%, but the final H_f worsens by 20.96% after DPO and mirroring. Thus, legalization quality alone is not a sufficient promotion criterion.

[3] In our tested Ariane133 N45 diagnostic runs, missing Diamond-sourceTopK averaged +1.52%/1.07×. First-iteration Level 1 guidance matched the case feature, selected it, and averaged −3.26%/0.71×, giving later rounds a local parent instead of spending early target iterations only on route discovery. Diamond-sourceTopK scores top-K DPO moves/swaps from a Diamond basin and passes hot nodes and segments to reorder. This diagnostic is not used as a separate evaluation target.

Table 5: Stage-local legalization counterexamples.

Case	Selected legalizer	Full-flow reference	H_{lg} / ref.	ΔH_{lg}	H_f / ref.	ΔH_f
AES dense N45	LEGALM	Diamond	137.8k / 138.8k	-0.76%	130.8k / 128.9k	+1.48%
JPEG dense N45	Negotiation	Negotiation	1.26M / 1.48M	-14.96%	711.7k / 588.3k	+20.96%
SWERV dense N45	Diamond	Negotiation	2.81M / 2.81M	-0.12%	2.71M / 2.71M	+0.02%

Table 6: Pattern-level hard cut-row repair evidence.

Case	Cut-Row Pattern	Fixed ref (D/N)	ReviewDSE	Evidence
Ariane133 N45	center-8um	fail/pass	pass, 42.2s	beats legal N by 1.03%, 18.9×
	center-9um	fail/fail	pass, 51.6s	strict repair; no fixed legal ref.
	center-10um	fail/TO	pass, 59.6s	strict repair; N reaches timeout
SWERV dense N45	center-5.25um	pass/fail	pass, 48.6s	beats legal fixed ref. by 3.69%, 3.4×
	center-5.5um	fail/fail	pass, 50.8s	strict repair; no fixed legal ref.
	center-6um	fail/fail	pass, 51.7s	strict repair; no fixed legal ref.
BPQUAD	center-5um	fail/TO	pass, 563.6s	D fails; N reaches 7200s cap
	center-6um	fail/TO	pass, 799.4s	D fails; N reaches 7200s cap
	center-8um	fail/TO	pass, 1085.0s	D fails; N reaches 7200s cap

D/N = Diamond/Negotiation; TO = 7200s timeout. center-$x$um is a vertical center-band cut-row with $x\mu$m halo. pass means OpenROAD and `check_placement` pass.

4.4 Hard Cut-Row Repair

Finally, we test legality recovery on hard cut-row stress tests. These layouts contain fragmented row topologies that cause fixed-source reference routes to fail legality, time out, or miss a strict legal solution. Unlike the nine-case QoR comparison, the objective here is to recover legal placements under the same protected legality checker and runtime cap.

Table 6 reports pattern-level replay evidence rather than only family-level counts. ReviewDSE produces strict legal placements on all nine hard cut-row patterns. For Ariane133 and SWERV, at least one pattern also has a legal fixed-source reference, and ReviewDSE improves both HPWL and runtime. For BPQUAD, the fixed routes fail or time out, so the evidence is legality recovery within 563.6–1085.0s rather than QoR dominance. These results complement Table 4 by showing that reviewed source-mechanism exploration can recover strict legality on failure-oriented stress tests, not only improve QoR on already feasible targets.

5 Discussion

ReviewDSE intentionally performs target-case source specialization, not universal optimizer replacement. Calibration provides method evidence and source starts, but each reported candidate is rebuilt, executed, and promoted only on its own target under the protected evaluator. The fixed measurement path prevents metric gaming, and promotion also requires canonical metrics, source/binary/metric consistency, runtime gating, and liveness evidence.

The main limitations are evaluator scope and search cost. Current results cover HPWL, runtime, displacement, and strict legality; timing, congestion, power, and signoff require additional protected metrics. The target searches average 2.15B logged tokens and 0.10B active uncached-input-plus-output tokens over 10 iterations, so ReviewDSE is most appropriate for high-value designs, hard failures, or cases where public knobs have limited remaining headroom.

6 Conclusion

ReviewDSE reframes open-source EDA tuning as protected white-box DSE: for a target design, bounded optimizer mechanisms can be

specialized under a fixed evaluator rather than only invoked through public knobs. In OpenROAD detailed placement, this source-level search boundary improves the post-DPL HPWL/runtime frontier beyond public-knob BO-DSE, reveals why stage-local winners require full-flow review, and recovers legality on hard cut-row stress tests. The main lesson is that source-mechanism exploration for EDA should not be treated as unrestricted tool rewriting: complete candidates must be judged by canonical metrics and legality checks, while reusable knowledge is accumulated as reviewed mechanism evidence.

References

[1] H. Geng, T. Chen, Q. Sun, and B. Yu, "Techniques for cad tool parameter auto-tuning in physical synthesis: a survey," in *2022 27th Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 2022, pp. 635–640.

[2] Y. Ma, Z. Yu, and B. Yu, "CAD tool design space exploration via bayesian optimization," in *2019 ACM/IEEE 1st Workshop on Machine Learning for CAD (MLCAD)*, 2019, pp. 1–6.

[3] The OpenROAD Project, https://theopenroadproject.org/, 2026.

[4] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont, P.-S. Huang, A. Z. Wagner, S. Shirobokov, B. Kozlovskii, F. J. Ruiz, A. Mehrabian *et al.*, "Alphaevolve: A coding agent for scientific and algorithmic discovery," *arXiv preprint arXiv:2506.13131*, 2025.

[5] H. Assumpção, D. Ferreira, L. Campos, and F. Murai, "Codeevolve: An open source evolutionary coding agent for algorithm discovery and optimization," *arXiv preprint arXiv:2510.14150*, 2025.

[6] C. Yu, R. Liang, C.-T. Ho, and H. Ren, "Autonomous code evolution meets np-completeness," *arXiv preprint arXiv:2509.07367*, 2025.

[7] X. Yao, J. Jiang, Y. Zhao, P. Liao, Y. Lin, and B. Yu, "EvoPlace: Evolution of Optimization Algorithms for Global Placement via Large Language Models," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2026.

[8] T. Jafri, "Gr-evolve: Design-adaptive global routing via llm-driven algorithm evolution," Master's thesis, Arizona State University, 2026.

[9] C. Yu and H. Ren, "Autonomous Evolution of EDA Tools: Multi-Agent Self-Evolved ABC," in *ACM/IEEE Design Automation Conference (DAC)*, 2026, pp. 1–6.

[10] A. Ghose, J. Jang, A. B. Kahng, and J. Lee, "Automated QoR improvement in OpenROAD with coding agents," *arXiv preprint arXiv:2601.06268*, 2026.

[11] A. Sohrabizadeh, C. H. Yu, M. Gao, and J. Cong, "AutoDSE: Enabling software programmers to design efficient fpga accelerators," *ACM Transactions on Design Automation of Electronic Systems*, vol. 27, no. 4, pp. 1–27, 2022.

[12] A. Grosnit, C. Malherbe, R. Tutunov, X. Wan, J. Wang, and H. Bou Ammar, "BOiLS: Bayesian optimisation for logic synthesis," in *2022 Design, Automation & Test in Europe Conference & Exhibition (DATE)*, 2022, pp. 1193–1196.

[13] A. Agnesina, K. Chang, and S. K. Lim, "VLSI placement parameter optimization using deep reinforcement learning," in *Proceedings of the 39th International Conference on Computer-Aided Design*, 2020, pp. 1–9.

[14] A. Agnesina, P. Rajvanshi, T. Yang, G. Pradipta, A. Jiao, B. Keller, B. Khailany, and H. Ren, "AutoDMP: Automated DREAMPlace-based macro placement," in *Proceedings of the 2023 International Symposium on Physical Design*, 2023, pp. 149–157.

[15] Y. Lu, S. Liu, Q. Zhang, and Z. Xie, "RTLLM: An open-source benchmark for design rtl generation with large language model," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, 2024, pp. 722–727.

[16] M. Liu, N. Pinckney, B. Khailany, and H. Ren, "VerilogEval: Evaluating large language models for verilog code generation," in *2023 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2023, pp. 1–8.

[17] S. Thakur, B. Ahmad, H. Pearce, B. Tan, B. Dolan-Gavitt, R. Karri, and S. Garg, "VeriGen: A large language model for verilog code generation," *ACM Transactions on Design Automation of Electronic Systems*, 2024.

[18] S. Liu, W. Fang, Y. Lu, J. Wang, Q. Zhang, H. Zhang, and Z. Xie, "RTLCoder: Fully open-source and efficient llm-assisted rtl code generation technique," 2023.

[19] S. Thakur, J. Blocklove, H. Pearce, B. Tan, S. Garg, and R. Karri, "AutoChip: Automating hdl generation using llm feedback," 2023.

[20] Z. Zhang, G. Chadwick, H. McNally, Y. Zhao, and R. Mullins, "LLM4DV: Using large language models for hardware test stimuli generation," 2023.

[21] R. Qiu, G. L. Zhang, R. Drechsler, U. Schlichtmann, and B. Li, "AutoBench: Automatic testbench generation and evaluation using llms for hdl design," in *2024 ACM/IEEE International Symposium on Machine Learning for CAD (MLCAD)*, 2024, pp. 1–10.

[22] Z. He, H. Wu, X. Zhang, X. Yao, S. Zheng, H. Zheng, and B. Yu, "ChatEDA: A large language model powered autonomous agent for eda," in *2023 ACM/IEEE 5th Workshop on Machine Learning for CAD (MLCAD)*, 2023, pp. 1–6.

[23] M. Liu, T.-D. Ene, R. Kirby, C. Cheng, N. Pinckney, R. Liang, J. Alben, H. Anand, S. Banerjee, I. Bayraktaroglu, B. Bhaskaran, B. Catanzaro, A. Chaudhuri, S. Clay, B. Dally, L. Dang, P. Deshpande, S. Dhodhi, S. Halepete, E. Hill, J. Hu, S. Jain, A. Jindal, B. Khailany, G. Kokai, K. Kunal, X. Li, C. Lind, H. Liu, S. Oberman, S. Omar, G. Pasandi, S. Pratty, J. Raiman, A. Sarkar, Z. Shao, H. Sun, P. P. Suthar, V. Tej, W. Turner, K. Xu, and H. Ren, "ChipNeMo: Domain-adapted llms for chip design," 2023.

[24] Y. Pu, Z. He, T. Qiu, H. Wu, and B. Yu, "Customized retrieval augmented generation and benchmarking for eda tool documentation qa," in *2024 IEEE/ACM International Conference on Computer-Aided Design (ICCAD)*, 2024.

[25] L. Shi, M. Kazda, B. Sears, N. Shropshire, and R. Puri, "Ask-EDA: A design assistant empowered by llm, hybrid rag and abbreviation de-hallucination," in *2024 IEEE LLM Aided Design Workshop (LAD)*, 2024.

A Artifact Appendix

Wenjie Yuan of Fudan University prepared and maintains the artifact package and its reproduction workflow.

A.1 Abstract

Our artifact contains the ReviewDSE implementation, the prompts and protected evaluator used in the experiments, and the source programs reported in the paper. It also includes a pinned open-source EDA environment and scripts for reproducing Tables 4–6 and Figures 4–5. The artifact runs on Rocky Linux 8.10 (x86-64) and requires authenticated Codex access. It does not require a GPU, a commercial EDA license, or a proprietary PDK. The evaluated release is archived on Zenodo under DOI 10.5281/zenodo.21629308, and the public repository is available at https://github.com/yuan-fd/DPLEvolve-AE.

A.2 Artifact Check-list (Meta-information)

- **Algorithm/program:** ReviewDSE applies protected Teacher–Student white-box DSE to OpenROAD detailed placement. Students edit bounded source code, and the Teacher reviews full-flow results on nine targets and nine cut-row patterns.
- **Tools/build/binary:** The artifact builds pinned OpenROAD, ORFS, Yosys, and OpenSTA versions with GCC/G++ 9 or later and CMake 3.16 or later. Each candidate has a private OpenROAD binary.
- **Models/access/prompts:** The paper used one `gpt-5.5` Teacher and four `gpt-5.4` Students; AE uses one `gpt-5.6-sol` Teacher and one `gpt-5.6-terra` Student. All use `xhigh` reasoning through authenticated Codex CLI access. The artifact supplies the prompts; other decoding settings retain the service defaults.
- **Data:** ORFS regenerates the Table 4 inputs. Three checksummed source snapshots support Table 5, and a separate ~200-MB DEF, Verilog, and SDC archive supports Table 6.
- **Environment/hardware:** The workflow requires Linux x86-64, Bash 4 or later, Make 4 or later, Python 3.11 or later, and the standard OpenROAD build dependencies. We tested it without a GPU on Rocky Linux 8.10 with two Xeon Platinum 8462Y+ processors (64 physical and 128 logical cores) and 314 GiB of RAM.
- **Execution/resources:** At least 100 GiB of free disk space is recommended. Setup takes several hours, and complete BO and model-search campaigns take several days; smaller machines may use less parallelism.
- **Metrics/output:** Runs report stage HPWL, strict legality, displacement, runtime, liveness, and source/binary/metric consistency. Scripts write fresh results, logs, review records, and figures.
- **Experiments/results:** The workflows reproduce Tables 4–6, Figures 4–5, selected-source replays, Level 1 calibration, and Level 2 search. Section A.6 gives the acceptance criteria.
- **Cost:** A paper run averaged 2.15B logged tokens (~0.10B active tokens) per target. Cost depends on the account and current provider prices.
- **Availability/licenses:** The public code uses the BSD 3-Clause License and contains no proprietary component. Third-party tools, data, and hosted models retain their own terms.
- **Workflow/archive:** GNU Make, Bash, Python, ORFS, and the Codex CLI manage the workflow. The evaluated archive has DOI 10.5281/zenodo.21629308.

A.3 Description

Access and dependencies. The Zenodo archive and live repository share the same entry points and integrity checks. The OpenROAD baseline follows the upstream `master` branch. The README gives the setup procedure, tested tool versions, and exact pinned revisions. Setup requires Linux x86-64, Git, rsync, the listed build packages, and network access, but neither root access nor commercial software.

Data and models. Only Table 5 input preparation changes core utilization: AES, JPEG, and SWERV use 70, 90, and 60, respectively. Its six roles draw from three archived implementations: AES uses LEGALM/Diamond, JPEG uses Negotiation/Negotiation, and SWERV uses Diamond/Negotiation. Diamond and Negotiation are OpenROAD implementations; the agent implemented LEGALM from the published method. Table 6 uses the paper's exact DEF, Verilog, and SDC inputs.

The paper search ran in April–May 2026. Because Codex does not expose a stable service revision, the harness records the model and reasoning effort; all other decoding settings retain their service defaults. A failed request is retried up to eight times with a 45-s linear backoff. The repository includes the prompts and their records, and model proposals remain stochastic.

A.4 Algorithm Flows

Algorithm 1 builds and freezes calibration evidence and source starts; Algorithm 2 uses that packet in a target-specific Teacher–Student loop. Both call the protected evaluator, and Level 2 cannot modify the packet.

Algorithm 1 Level 1 calibration and warm-start construction.

Input: Calibration cases $\mathcal{C}$, source O, evaluator E, source starts $\mathcal{B}_0$, S_C Students
Output: Frozen packet $W = (K, \mathcal{B})$

$K \leftarrow \emptyset$; $\mathcal{B} \leftarrow \mathcal{B}_0$
for all $c \in \mathcal{C}$ **do**
  $b_c \leftarrow \texttt{Evaluate}(E, O, c)$; $Q_c \leftarrow \texttt{TeacherPlan}(c, b_c, K, \mathcal{B})$
  for all $s \in \{1, \ldots, S_C\}$ **in parallel do**
    $O_s \leftarrow \texttt{Instantiate}(\mathcal{B}, Q_c[s])$
    $\Delta_s \leftarrow \texttt{StudentEdit}(O_s, Q_c[s])$
    $\widehat{O}_s \leftarrow \texttt{Build}(O_s + \Delta_s)$
    $a_s \leftarrow \texttt{Evaluate}(E, \widehat{O}_s, c)$
  end for
  $(K, \mathcal{B}) \leftarrow \texttt{TeacherReview}(\{a_s\}, K, \mathcal{B})$
end for
return $W \leftarrow \texttt{Freeze}(K, \mathcal{B})$

Each Student returns its complete source and diff with full-flow metrics, legality, runtime, and liveness evidence. The Teacher reviews these results and freezes useful evidence and source starts. Calibration uses AES, JPEG, and SWERV at 70, 90, and 60 utilization, respectively.

During target search, Students work in private source trees while W remains read-only. Promotion requires successful build, legality, canonical metric, source-to-binary consistency, liveness, and runtime checks. The Teacher stores other useful outcomes in target-local history L_d without changing W.

Algorithm 2 Level 2 Teacher–Student target search.

Input: Target d, source O, evaluator E, frozen packet W, S Students, R iterations, runtime gate g
Output: Promoted candidate p_d and target-local record L_d

$b_d \leftarrow \texttt{Evaluate}(E, O, d)$; $L_d \leftarrow \{b_d\}$; $p_d \leftarrow O$
$\texttt{CreatePrivateWorkspaces}(S, O)$
for $r = 1$ to R **do**
  $Q_r \leftarrow \texttt{TeacherPlan}(d, b_d, W, L_d, p_d)$
  for all $s \in \{1, \ldots, S\}$ **in parallel do**
    $O_s \leftarrow \texttt{Instantiate}(p_d, W, Q_r[s])$
    $\Delta_s \leftarrow \texttt{StudentEdit}(O_s, Q_r[s])$
    $\widehat{O}_s \leftarrow \texttt{Build}(O_s + \Delta_s)$
    $a_s \leftarrow \texttt{Evaluate}(E, \widehat{O}_s, d)$
  end for
  $V_r \leftarrow \texttt{ContractValid}(\{a_s\}, g)$
  $L_d \leftarrow L_d \cup \texttt{TeacherReview}(\{a_s\}, V_r)$
  $p_d \leftarrow \texttt{PromoteBest}(V_r, p_d)$; $\texttt{PreparePeerEvidence}(L_d)$
end for
return (p_d, L_d) ▷ W remains frozen

A.5 Installation and Workflow

From the artifact root, prepare the pinned environment and verify the toolchain and model access as follows:

```
make doctor
make bootstrap
make build-tools THREADS=8
make check
make prepare-paper-inputs CASE=aes_nangate45 THREADS=8
make validate-evaluator CASE=aes_nangate45 THREADS=8
make check-demo-models
```

`make check-demo-models` sends one small request to each model and must report `MODEL_READY` for both roles. This verifies model access; the functional evaluation does not require a live closed-loop search. Reproduce the reported experiments as follows:

```
bash artifacts/01-table4-qor/reproduce.sh \
  --threads 10
make check-table5-data
make reproduce-table5 THREADS=10
bash artifacts/03-table6-cutrow/reproduce.sh \
  --fetch --threads 10
bash artifacts/04-figures/reproduce.sh
make reproduce-paper-search \
  ACKNOWLEDGE_LLM_COST=yes THREADS=10
```

The reproduction workflow runs nine defaults, 400 BO trials per target, 18 selected-program replays, six Table 5 roles, 27 cut-row cases, and figure regeneration with the required Teacher and Student configuration. The final command starts a new stochastic search and requires explicit acknowledgement of its model cost; this paper-scale run is not part of the functional check.

A.6 Evaluation and Expected Results

The scripts derive observed metrics from fresh runs before comparing them with the references. No expected value is copied into an observed field.

- **Table 4:** A full rerun produces nine legal cases. Mean HPWL changes are −0.38% for BO, −1.78% for HPWL selection, and −1.68% for GHR selection; ReviewDSE runtime ratios are 1.34× and 1.11×. Tolerances are 0.06 percentage points for HPWL and 0.20 for runtime ratios.
- **Table 5:** Each of the six rebuilt selected/reference roles must satisfy $\Delta H_{lg} < 0$ and $\Delta H_f > 0$. AES, JPEG, and SWERV report −0.76%/+1.48%, −14.96%/+20.96%, and −0.12%/+0.02%, respectively.
- **Table 6:** A full rerun produces 27 rows whose status and strict-legality classifications must match exactly; runtime may vary by 35%. All nine ReviewDSE cases, one Diamond case, and one Negotiation case must be legal.
- **Figures 4–5:** Figure 4 must contain 96 observed points and mark the three unavailable SWERV points without imputation. Figure 5 must recompute the runtime-ratio Pareto set; the workflow writes fresh TSV and SVG files.

The public Level 1 profile reconstructs calibration because the original Student breadth and frozen packet were not retained. Fresh searches are judged from protected results and trajectories, not by reproducing the same edits.

A.7 Integrity and Failure Reporting

Run the following checks before reporting any result:

```
make test
make validate-configs
make zenodo-audit
```

The audit checks all three Table 5 source snapshots, and Table 6 starts only after its download passes the integrity check. Scripts write new results and use references only for comparison. A failed build, legality check, metric contract, or tolerance returns a nonzero status and preserves the command, run record, result directory, and first failure.

A.8 Experiment Customization

The `CASE`, `THREADS`, `STUDENTS`, and `ITERATIONS` variables select the target, concurrency, and search size; separate variables select both models. A new campaign requires a distinct `DSE_RUN_PREFIX`. Selection retains the paper's 2× runtime gate, and paper-scale model runs require the cost acknowledgement above.

A.9 Methodology

The artifact follows the current ACM badging policy, the cTuning submission guide, and the MLCAD AE instructions.